

Influo AI — Complete Product Specification

AI-Powered Brand Deal Management for Content Creators

Table of Contents

1. Executive Summary
 2. Problem Analysis
 3. Solution Overview
 4. User Journey & Workflows
 5. Feature Specifications
 6. Technical Architecture
 7. Data Models
 8. AI System Design
 9. Security & Privacy
 10. Success Metrics
-

1. Executive Summary

What is Influo AI?

Influo AI is a SaaS platform that transforms how content creators manage brand collaborations by automatically detecting brand deal emails, generating AI-powered responses, and tracking deals through a visual pipeline.

Core Value Proposition

"Turn brand deal emails into a revenue pipeline — automatically."

Instead of:

- Manually scanning Gmail for brand opportunities
- Writing replies from scratch each time
- Tracking deals in spreadsheets that go stale
- Chasing payments via WhatsApp reminders

Creators get:

- Automatic brand email detection and extraction
- One-click AI reply generation (accept/negotiate/decline)
- Visual deal pipeline with payment tracking
- Revenue insights and deal history

Target User

Primary: Instagram/YouTube creators with 10k-200k followers actively working with brands (3-10 deals/month)

Secondary: Micro-influencers scaling from 1-2 deals to consistent brand work

2. Problem Analysis

Problem 1: Brand Emails Lost in Inbox Chaos

Current Painful Reality

A creator with 50k Instagram followers gets:

- 200+ emails/day (mix of fan mail, spam, newsletters, brand inquiries)
- Brand collaboration emails land in Promotions, Spam, or buried in Primary
- Subject lines are vague: "Collaboration Opportunity" "Partnership Inquiry" "Let's Work Together"
- Creator manually searches "collaboration OR sponsorship OR brand" every few days
- Misses 30-40% of legitimate brand opportunities

Why This Happens

- Gmail's filters can't distinguish brand deals from sales pitches
- No structured way to identify collaboration intent
- Creators don't have time to read 200 emails/day
- PR agencies use generic templates that look like spam

Quantified Impact

- **Lost Revenue:** ₹15k-₹50k/month from missed opportunities
 - **Time Waste:** 2-3 hours/week manually searching inbox
 - **Mental Load:** Constant anxiety about "what if I missed something?"
-

Problem 2: Inefficient Deal Communication

Current Painful Reality

When a creator receives a brand email:

```
From: partnerships@beautybrand.com
Subject: Collaboration Opportunity

Hi [Creator Name],

We love your content! We'd like to collaborate with you
for our new skincare line launch. Are you interested?

Looking forward to hearing from you.
Best,
Marketing Team
```

What happens next:

1. Creator reads email (5 min)
2. Googles brand to check legitimacy (10 min)
3. Checks Instagram for similar creator collaborations (15 min)
4. Writes reply asking for budget, deliverables, timeline (20 min)
5. Waits 3-7 days for response
6. Negotiates back-and-forth (3-5 email exchanges over 2 weeks)

Total time per deal: 4-6 hours

For a creator handling 5 deals/month: 20-30 hours of email tennis

Why This Happens

- Every brand email is different (no standardization)
- Creators don't know what to ask upfront
- No templates for common scenarios (accept/negotiate/decline)
- Each reply is written from scratch

Quantified Impact

- **Time Waste:** 20-30 hours/month on email communication
- **Deal Slippage:** 20% of interested brands ghost after slow response
- **Undercharging:** Creators accept first offer due to negotiation fatigue

Problem 3: No Deal Visibility or Structure

Current Painful Reality

A creator with 8 active brand deals tracks them like this:

Method A: Gmail Starred Emails

- Stars brand emails
- No way to see deal stage (negotiating? accepted? completed?)
- Payment status unknown until creator manually checks bank

Method B: Google Sheets

Brand Name	Amount	Status	Payment Date	Notes
Nike	₹25k	Done	???	Check with them
Nykaa	₹15k	???	???	Need to reply
Boat	₹30k	Pending	15-Dec	Sent invoice

Problems:

- Sheet is never updated in real-time
- No link between email thread and sheet row
- Status is ambiguous ("Done" but not paid?)
- No reminders for payment follow-ups

Method C: Mental Tracking

- "I think Nike paid me last week... or was it two weeks ago?"
- Forgets to follow up on pending payments
- Can't answer "How much have I earned this month?" without manual calculation

Why This Happens

- No tool designed for creator deal workflows
- CRMs (Notion, Airtable) are too generic and require manual entry
- Email and deal tracking are disconnected systems

Quantified Impact

- **Lost Revenue:** ₹10k-₹30k/month from forgotten payment follow-ups

- **Time Waste:** 3-5 hours/month updating spreadsheets manually
 - **Stress:** No clear view of income, pending work, or payment status
-

Problem 4: Pricing Opacity and Undercharging

Current Painful Reality

Brand email says:

┆ "What's your rate for 1 Instagram Reel + 2 Stories?"

Creator's internal panic:

- "What do other creators my size charge?"
- "Is ₹20k too high? Too low?"
- "Should I quote ₹15k to be safe?"
- *Quotes ₹15k, brand accepts immediately* → undercharged by ₹10k

Industry Reality:

- A 50k follower creator with 5% engagement can charge ₹20k-₹35k for a reel
- Creators with no pricing data consistently undercharge by 30-50%
- Brands exploit this information asymmetry

Why This Happens

- No transparent creator pricing database in India
- Influencer marketing agencies keep pricing data private
- Creators don't share rates publicly (competitive fear)
- Engagement rate impact on pricing is unclear

Quantified Impact

- **Lost Revenue:** ₹50k-₹2L/year from systematic undercharging
 - **Confidence Gap:** Creators avoid negotiating due to uncertainty
-

3. Solution Overview

How Influo AI Solves Each Problem

Problem	Influo AI Solution	User Outcome
Brand emails lost in chaos	AI auto-detects collaboration emails, extracts them to dedicated inbox	Zero missed opportunities, 2 hours/week saved
Inefficient communication	AI generates context-aware replies (accept/negotiate/decline) in 1 click	20 hours/month saved, faster deal closure
No deal visibility	Visual pipeline tracks every deal from inquiry → payment	Always know what's pending, what's paid, what's overdue
Pricing uncertainty	Benchmark pricing suggestions based on follower count + engagement	Charge fair rates, negotiate with confidence

4. User Journey & Workflows

4.1 Onboarding Flow

Step 1: Sign Up (2 minutes)

Screen: Landing page → "Start Free Trial"

User Actions:

1. Clicks "Continue with Google"
2. Google OAuth consent screen appears
3. Grants permissions:
 - Read Gmail messages
 - Send emails on your behalf
 - Manage labels

Backend Actions:

1. Creates user account in PostgreSQL
2. Stores OAuth tokens (encrypted)
3. Redirects to dashboard

Success State: User sees empty inbox with message:

| "Scanning your Gmail for brand collaboration emails... This may take 2-3 minutes."

Step 2: Initial Gmail Sync (3-5 minutes)

What Happens:

1. Background worker fetches last 3 months of emails (API batch request)
2. AI classifier scans each email subject + body
3. Identifies brand collaboration emails using keyword patterns + LLM classification
4. Extracts structured data (brand name, offer type, budget if mentioned)
5. Populates "Brand Deals Inbox"

User Experience:

- Progress bar: "Analyzing 2,847 emails... Found 12 brand opportunities"
- When complete, shows list of detected brand emails

Edge Case Handling:

- If 0 brand emails found: "No collaboration emails detected. Try connecting Instagram DM access (coming soon) or wait for new emails."
 - If 50+ detected: "Found 50+ brand emails. Showing most recent 20. View all →"
-

Step 3: Profile Setup (2 minutes)

Required Fields:

- Primary platform (Instagram / YouTube / Both)
- Instagram handle
- Follower count (fetched via Instagram Graph API if connected)
- Average engagement rate (calculated or manual input)

Optional Fields:

- Content niche (Beauty, Tech, Lifestyle, Fashion, Food, Fitness, Travel)
- Typical deliverables (Reels, Stories, Posts, YouTube integrations)
- Preferred brand categories (willing to work with)

Why This Matters:

- Pricing engine uses follower count + engagement for rate suggestions
 - Deal categorization improves with niche context
 - Better AI reply personalization
-

4.2 Core Workflow: Brand Email → Deal Closure

Scenario: Creator Receives New Brand Email

Email Content:

```
From: marketing@myntra.com
Subject: Collaboration Proposal - Fashion Week Campaign

Hi Priya,

We're launching our Spring Fashion Week collection and would love to partner
with you for a dedicated campaign.

Deliverables:
- 2 Instagram Reels (product showcase)
- 3 Stories (behind-the-scenes)
- 1 carousel post

Timeline: Shoot by Feb 20, post by Feb 25

Budget: We're flexible based on your rates. Please share your proposal.

Looking forward to working together!

Best,
Anjali - Myntra Creator Partnerships
```

Step 1: Email Detection & Extraction (Automatic)

Backend Process:

1. Gmail Webhook Trigger:

- Gmail API sends push notification: "New email from marketing@myntra.com"
- Webhook hits: `POST /api/webhooks/gmail`

2. AI Classification:

python

```
# Pseudo-code
email_data = gmail_api.get_message(message_id)

classification = openai.classify(
    subject=email_data.subject,
    body=email_data.body,
    sender=email_data.from,
    prompt="Is this a brand collaboration email? YES/NO"
)

if classification == "YES":
    extracted_data = openai.extract_entities(
        email_body=email_data.body,
        schema={
            "brand_name": "string",
            "offer_type": "enum[Paid Sponsorship, Barter, Affiliate, Gifted]",
            "deliverables": "list[string]",
            "budget_mentioned": "boolean",
            "budget_amount": "number or null",
            "deadline": "date or null"
        }
    )
```

3. Data Stored in Database:

json

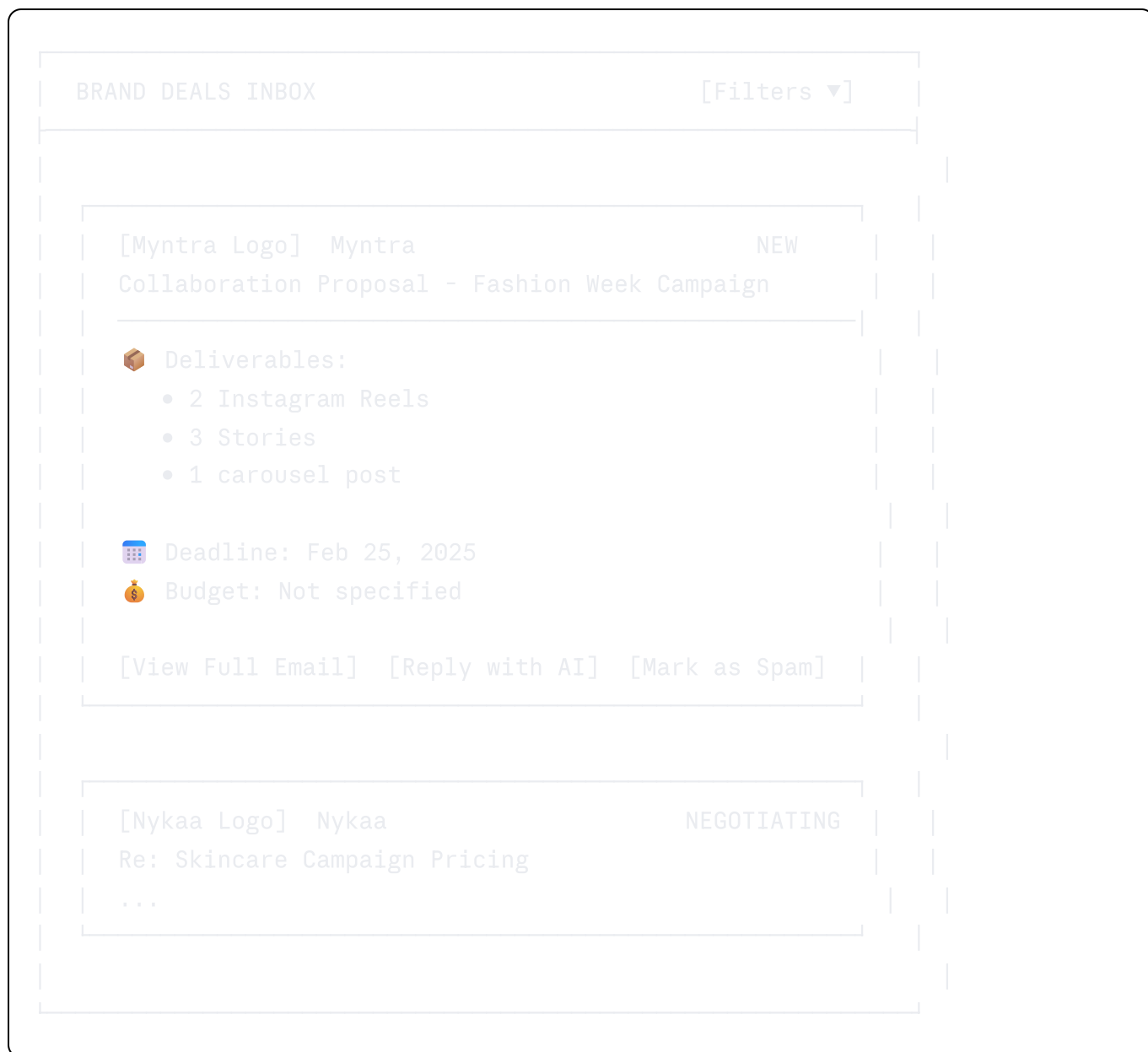
```
{
  "deal_id": "deal_67890",
  "user_id": "user_12345",
  "brand_name": "Myntra",
  "sender_email": "marketing@myntra.com",
  "subject": "Collaboration Proposal - Fashion Week Campaign",
  "received_at": "2025-02-10T14:30:00Z",
  "stage": "new",
  "extracted_data": {
    "offer_type": "Paid Sponsorship",
    "deliverables": [
      "2 Instagram Reels",
      "3 Stories",
      "1 carousel post"
    ],
    "budget_mentioned": false,
    "deadline": "2025-02-25"
  },
  "gmail_thread_id": "thread_abc123",
  "gmail_message_id": "msg_xyz789"
}
```

User Experience:

- Desktop notification: "New brand deal from Myntra"
- Email appears in "Brand Deals Inbox" with label: **NEW**
- Deal card shows:
 - Brand logo (fetched from Clearbit API)
 - Extracted deliverables
 - "Reply with AI" button

Step 2: User Reviews Email in Influo AI Inbox

UI Layout:



User Clicks: [Reply with AI]

Step 3: AI Reply Generation Modal

UI:

Generate AI Reply

[X]

What do you want to do?

○ Accept & Share Pricing

○ Negotiate Terms

○ Request More Details

○ Politely Decline

Your Pricing (Auto-calculated)

Based on 52k followers, 6.2% engagement

2 Reels @ ₹18,000 each

= ₹36,000

3 Stories @ ₹3,000 each

= ₹9,000

1 Carousel Post

= ₹12,000

Total Package Price

= ₹57,000

[Edit Pricing]

Additional Instructions (Optional)

Mention that I need 50% advance payment

[Generate Reply]

User Selects: "Accept & Share Pricing"

User Edits Pricing: Changes Reel price to ₹20k each (Total = ₹61k)

User Adds Note: "Mention 50% advance payment"

User Clicks: [Generate Reply]

Step 4: AI Reply Generated

Backend Process:

python

AI Reply Generation Logic

```
user_profile = get_user_profile(user_id)
deal_data = get_deal(deal_id)
pricing_input = request.pricing # User's edited pricing

system_prompt = f"""
You are a professional brand collaboration manager for {user_profile.name},
a content creator with {user_profile.followers} followers on Instagram.

Generate a professional, warm email reply to this brand collaboration proposal.

CONTEXT:
- Brand: {deal_data.brand_name}
- Deliverables: {deal_data.deliverables}
- Deadline: {deal_data.deadline}

TONE:
- Professional but friendly
- Confident (not desperate)
- Clear and structured

PRICING TO QUOTE:
{pricing_input.breakdown}
Total: ₹{pricing_input.total}

ADDITIONAL REQUIREMENTS:
- {user_custom_instructions}

EMAIL STRUCTURE:
1. Thank them for the opportunity
2. Express genuine interest in the campaign
3. List deliverables clearly
4. Present pricing breakdown
5. Mention payment terms (50% advance)
6. Suggest next steps (call/meeting)
7. Professional sign-off
"""

ai_reply = openai.chat.completions.create(
    model="gpt-4",
    messages=[
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": deal_data.original_email}
```

```
],  
    temperature=0.7  
)  
  
return ai_reply.choices[0].message.content
```

Generated Email:

Subject: Re: Collaboration Proposal - Fashion Week Campaign

Hi Anjali,

Thank you for reaching out! I'm excited about Myntra's Spring Fashion Week collection and would love to collaborate on this campaign.

Here's my proposal for the deliverables:

 Instagram Content Breakdown:

- 2 Instagram Reels (product showcase) - ₹20,000 each
- 3 Instagram Stories (BTS + swipe-ups) - ₹3,000 each
- 1 Carousel Post (styling guide) - ₹12,000

 Total Campaign Investment: ₹61,000

This includes:

- ✓ Concept development and scripting
- ✓ Professional shoot setup
- ✓ Editing and posting at optimal times
- ✓ Usage rights for 6 months

 Payment Terms:

50% advance upon agreement, 50% upon content delivery

 Timeline:

I can commit to shooting by Feb 20 and posting by Feb 25 as requested.

I'd love to jump on a quick call to discuss the campaign vision and any specific brand guidelines. Does this week work for you?

Looking forward to creating something amazing together!

Best,

Priya

Instagram: @priya.create | 52k followers

UI Shows:

AI-Generated Reply

[Edit] [Regenerate]

[Full email content shown above]

☒ Save this deal to pipeline as "Negotiating"

[Send Now]

[Save as Draft]

[Copy to Clipboard]

User Reviews: Reads generated reply, makes minor edit (changes "excited" to "thrilled")

User Clicks:

[Send Now]

Step 5: Email Sent & Deal Moves to Pipeline

Backend Actions:

1. Send Email via Gmail API:

```
python

gmail_api.send_message(
    thread_id=deal_data.gmail_thread_id,
    to=deal_data.sender_email,
    subject=f"Re: {deal_data.subject}",
    body=ai_reply_edited,
    from_email=user_profile.email
)
```

2. Update Deal Status:

```
sql
```



```
UPDATE deals
SET stage = 'negotiating',
    quoted_amount = 61000,
    last_replied_at = NOW()
WHERE deal_id = 'deal_67890';
```

3. Create Activity Log:

json

```
{
  "activity_id": "activity_999",
  "deal_id": "deal_67890",
  "type": "email_sent",
  "timestamp": "2025-02-10T15:45:00Z",
  "metadata": {
    "reply_mode": "accept_and_price",
    "ai_generated": true,
    "user_edited": true,
    "quoted_amount": 61000
  }
}
```

User Experience:

- Success toast: "✓ Reply sent to Myntra"
- Deal card moves from "NEW" inbox to "NEGOTIATING" column in pipeline
- Email thread is auto-labeled in Gmail: Influo AI/Negotiating

Step 6: Brand Responds (Accept/Counter/Reject)

Scenario A: Brand Accepts

From: marketing@myntra.com
Subject: Re: Collaboration Proposal - Fashion Week Campaign

Hi Priya,

Perfect! Your pricing works for us. Let's proceed.

I'll send over the contract and brief by tomorrow.
Can you share your bank details for the 50% advance?

Best,
Anjali

Influ AI Detects:

- Keyword match: "pricing works" "let's proceed"
- AI classification: ACCEPTANCE
- Suggested action: Move to "ACCEPTED" stage

User Sees:

 Myntra accepted your proposal!

Suggested Amount: ₹61,000

[\[Move to Accepted\]](#) [\[View Email\]](#)

User Clicks: [\[Move to Accepted\]](#)

Backend Updates:

```
sql

UPDATE deals
SET stage = 'accepted',
    final_amount = 61000,
    accepted_at = NOW()
WHERE deal_id = 'deal_67890';
```

User Sees in Pipeline: Deal card now shows in "ACCEPTED" column with:

- Brand: Myntra

- Amount: ₹61,000
 - Deliverables: 2 Reels, 3 Stories, 1 Post
 - Deadline: Feb 25
 - Status: Awaiting advance payment
-

Scenario B: Brand Counters

From: marketing@myntra.com

Hi Priya,

Thanks for the detailed proposal! Your content is great.

Our budget for this campaign is ₹50,000. Would that work for you?

Best,
Anjali

Influ AI Detects:

- Keyword: "budget is ₹50,000"
- AI classification: COUNTER_OFFER
- Extracted counter: ₹50,000 (₹11k below quoted price)

User Sees:

Myntra sent a counter-offer

Your Quote: ₹61,000

Their Counter: ₹50,000 (18% lower)

[Accept Counter] [Negotiate Further] [Decline]

User Clicks: [Negotiate Further]

AI Reply Modal Opens:

Pre-filled suggestion:

Hi Anjali,

I appreciate you getting back! I'd love to make this work.

Given the scope (2 Reels + 3 Stories + 1 Post), my best price would be ₹55,000. This includes usage rights for 6 months.

I can also offer ₹50,000 if we reduce to:

- 2 Reels + 2 Stories (removing the carousel post)

Let me know what works best for the campaign!

Best,

Priya

User edits, sends.

Step 7: Deal Completion & Payment Tracking

After Content is Posted:

User Manually Updates:

- Clicks on deal card in "ACCEPTED" column
- Clicks [Mark as Completed]

Backend Updates:

```
sql

UPDATE deals
SET stage = 'completed',
    completed_at = NOW(),
    payment_status = 'pending'
WHERE deal_id = 'deal_67890';
```

User Sees: Deal moves to "COMPLETED" column with tag:  Payment Pending

When Payment Received:

User Manually Updates:

- Clicks [Mark as Paid]

- Optionally adds payment date

Backend Updates:

```
sql

UPDATE deals
SET stage = 'paid',
    payment_status = 'paid',
    paid_at = '2025-02-28'
WHERE deal_id = 'deal_67890';

-- Update user earnings
UPDATE user_stats
SET total_earnings = total_earnings + 61000,
    deals_closed = deals_closed + 1
WHERE user_id = 'user_12345';
```

User Sees:

- Deal card shows: ✔ Paid on Feb 28
 - Dashboard earnings widget updates: ₹61k earned this month
-

Payment Reminder Feature:

If deal is in "COMPLETED" stage for 7+ days without payment:

Influo AI Shows:

⚠ Payment overdue from Myntra (7 days past completion)

Amount Due: ₹30,500 (50% balance)

[\[Send Payment Reminder\]](#) [\[Mark as Paid\]](#) [\[Report Issue\]](#)

If User Clicks [Send Payment Reminder]:

AI generates polite follow-up:

Subject: Following up - Payment for Fashion Week Campaign

Hi Anjali,

Hope you're doing well! The Fashion Week campaign content performed really well (500k+ views across all posts).

I wanted to follow up on the pending 50% balance payment of ₹30,500 for the campaign.

Could you share an expected timeline for processing?

Thanks!

Best,

Priya

4.3 Deal Pipeline View

Visual Layout:

DEALS PIPELINE			Total: ₹2.4L (8 deals)
NEW (2)	NEGOTIATING (3)	ACCEPTED (1)	COMPLETED (2)
Boat ₹25k 2 Reels	Nykaa ₹40k Counter	Myntra ₹61k Advance sent	Nike ₹35k  Pay Pending
Mamaearth Barter PR Box	Sugar ₹30k Waiting reply		Bewakoof ₹18k  Paid
	Plum ₹22k Sent pricing		

Deal Card (Expanded View):

[Myntra Logo] Myntra Fashion Week Campaign

Status: ACCEPTED

Amount: ₹61,000 (₹30,500 advance received ✓)

 Deliverables:

- 2 Instagram Reels
- 3 Instagram Stories
- 1 Carousel Post

 Timeline:

Shoot by: Feb 20

Post by: Feb 25

 Contact:

Anjali - marketing@myntra.com

 Activity History:

- Feb 10: Initial email received
- Feb 10: Sent pricing (₹61k)
- Feb 11: Brand accepted
- Feb 12: Advance payment received (₹30.5k)

[View Email Thread] [Mark as Completed] [Add Note]

5. Feature Specifications

Feature 1: Smart Brand Deal Inbox

What It Does

Automatically detects brand collaboration emails from Gmail and displays them in a dedicated inbox separate from regular emails.

How It Works

1. Gmail Integration:

- User grants OAuth permissions (read emails, send emails, manage labels)
- Backend subscribes to Gmail Push Notifications via Pub/Sub
- On new email: Gmail sends webhook → Influx AI processes

2. AI Email Classification:

```
python
```

```

def classify_brand_email(email_data):
    """
    Determines if email is a brand collaboration opportunity
    """

    # Quick keyword filter (saves AI API calls)
    quick_keywords = [
        "collaboration", "partnership", "sponsorship",
        "brand deal", "influencer", "creator",
        "campaign", "paid promotion"
    ]

    subject_body = f"{email_data.subject} {email_data.body_preview}".lower()

    if not any(keyword in subject_body for keyword in quick_keywords):
        return {"is_brand_email": False}

    # AI classification for edge cases
    prompt = f"""
    Analyze this email and determine if it's a brand collaboration request.

    FROM: {email_data.sender_email}
    SUBJECT: {email_data.subject}
    BODY: {email_data.body_preview[:500]}

    Return JSON:
    {{
        "is_brand_email": true/false,
        "confidence": 0-100,
        "reasoning": "brief explanation"
    }}

    Brand emails typically:
    - Come from brands, PR agencies, or marketing teams
    - Propose paid sponsorships, barter, or gifted products
    - Ask about rates, availability, or interest in collaboration

    NOT brand emails:
    - Job offers from companies
    - Sales pitches selling software/services to the creator
    - Fan emails
    - Newsletter subscriptions
    """

```

```
result = openai.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": prompt}],
    response_format={"type": "json_object"}
)

return json.loads(result.choices[0].message.content)
```

3. Data Extraction:

python

```

def extract_deal_data(email_body):
    """
    Extracts structured data from brand email
    """

    extraction_prompt = f"""
    Extract the following information from this collaboration email:

    EMAIL:
    {email_body}

    Return JSON with these fields:
    {{
        "brand_name": "string (company name)",
        "offer_type": "Paid Sponsorship | Barter | Gifted Product | Affiliate",
        "deliverables": ["list of requested content"],
        "budget_mentioned": true/false,
        "budget_amount": number or null,
        "budget_currency": "INR" or null,
        "deadline": "YYYY-MM-DD" or null,
        "sender_name": "contact person name",
        "sender_role": "their job title if mentioned"
    }}

    If information is not present, use null.
    """

    result = openai.chat.completions.create(
        model="gpt-4o",
        messages=[{"role": "user", "content": extraction_prompt}],
        response_format={"type": "json_object"}
    )

    return json.loads(result.choices[0].message.content)

```

User-Facing Features

Inbox View:

- List of all brand collaboration emails
- Each card shows: Brand logo, subject, deliverables, budget (if mentioned)
- Filter by: Date, Status (New/Replied/Archived), Offer Type
- Search by brand name

Email Actions:

- View full email thread
- Reply with AI
- Mark as not a brand email (improves AI training)
- Archive (hide from inbox but keep in pipeline)

Edge Cases

Scenario	Handling
Brand email in Spam folder	Backend scans Spam folder too
Email thread (multiple back-and-forth)	Groups by thread_id, shows latest message
False positive (non-brand email detected)	User marks as "Not a brand email" → retrains model
False negative (brand email missed)	User manually adds via "Import from Gmail" button

Feature 2: AI Reply Generation

What It Does

Generates contextually appropriate email responses based on user intent (accept, negotiate, decline, request details).

Reply Modes

Mode 1: Accept & Share Pricing

Input:

- Deal deliverables (extracted from email)
- User's follower count + engagement rate
- User's custom pricing (if they edit auto-suggestions)

Output: Professional email that:

- Thanks brand for opportunity
- Lists deliverables clearly
- Presents pricing breakdown (itemized)
- States payment terms (advance %)

- Suggests next steps (call/meeting)

Example Pricing Calculation:

python

```
def calculate_pricing(user_profile, deliverables):
    """
    Suggests pricing based on creator stats
    """

    # Base rates (per 10k followers, adjusted for engagement)
    base_rates = {
        "instagram_reel": 3500,      # ₹3500 per 10k followers
        "instagram_story": 800,      # ₹800 per 10k followers
        "instagram_post": 2500,      # ₹2500 per 10k followers
        "youtube_integration": 8000   # ₹8000 per 10k subscribers
    }

    follower_multiplier = user_profile.followers / 10000

    # Engagement rate boost
    if user_profile.engagement_rate > 5:
        engagement_boost = 1.3
    elif user_profile.engagement_rate > 3:
        engagement_boost = 1.15
    else:
        engagement_boost = 1.0

    pricing = {}
    for deliverable in deliverables:
        content_type = map_deliverable_to_type(deliverable)
        base_price = base_rates.get(content_type, 5000)

        suggested_price = base_price * follower_multiplier * engagement_boost
        pricing[deliverable] = round(suggested_price, -2) # Round to nearest ₹

    return pricing

# Example:
# User: 50k followers, 6% engagement
# Deliverable: "2 Instagram Reels"
# Calculation: ₹3500 * 5 * 1.3 = ₹22,750 per reel
```

Mode 2: Negotiate Terms

Use Case: Brand's offer is below expected rate, or deliverables are unclear.

Output:

- Politely acknowledges brand's message
 - Explains value proposition (engagement stats, past performance)
 - Presents counter-offer with justification
 - Offers alternative package (reduce scope for lower price)
-

Mode 3: Request More Details

Use Case: Email lacks clarity on budget, timeline, or deliverables.

Output:

- Thanks for reaching out
 - Asks specific questions:
 - "What's your budget for this campaign?"
 - "What are the exact deliverables?"
 - "What's the content timeline?"
 - "Will there be usage rights or exclusivity?"
-

Mode 4: Politely Decline

Use Case: Brand doesn't align with creator's niche, or offer is too low.

Output:

- Thanks for considering them
 - Explains why it's not a fit (schedule conflict, niche mismatch, budget constraints)
 - Leaves door open for future collaboration
 - Professional tone (maintains relationship)
-

Technical Implementation

AI Prompt Structure:

python


```

def generate_reply(mode, deal_data, user_profile, custom_instructions):

    system_prompt = f"""
    You are a professional brand collaboration manager for {user_profile.name}.

    Creator Profile:
    - Name: {user_profile.name}
    - Platform: Instagram
    - Followers: {user_profile.followers}
    - Engagement Rate: {user_profile.engagement_rate}%
    - Niche: {user_profile.niche}

    TONE GUIDELINES:
    - Professional but warm (not corporate or robotic)
    - Confident (not desperate or overly eager)
    - Clear and direct (avoid fluff)
    - Use first person ("I" not "we")

    EMAIL STRUCTURE:
    1. Greeting (use sender's name if available)
    2. Opening (thank them / acknowledge their message)
    3. Main content (based on mode)
    4. Next steps / Call to action
    5. Sign-off

    AVOID:
    - Generic phrases ("I hope this email finds you well")
    - Excessive exclamation marks
    - Desperation ("I'd love to work with any brand!")
    - Underselling ("I'm just a small creator")
    """

    if mode == "accept_and_price":
        user_prompt = f"""
        Generate a reply to this brand collaboration email:

        ORIGINAL EMAIL:
        {deal_data.original_email}

        DELIVERABLES:
        {deal_data.deliverables}

        YOUR PRICING:
        {format_pricing_breakdown(deal_data.pricing)}

```

```

    ADDITIONAL INSTRUCTIONS:
    {custom_instructions}

    The email should:
    - Express genuine interest in the campaign
    - Present pricing breakdown clearly
    - Mention payment terms: 50% advance, 50% on completion
    - Include usage rights: 6 months unless negotiated
    - Suggest a call to discuss campaign details
    """

elif mode == "negotiate":
    user_prompt = f"""
    The brand offered ₹{deal_data.brand_offer}, but you want ₹{deal_data.target_offer}.

    Generate a negotiation email that:
    - Acknowledges their offer
    - Explains your value (engagement, quality, audience fit)
    - Presents your counter-offer with justification
    - Offers alternative: reduced scope for their budget
    - Keeps tone collaborative, not confrontational
    """

# ... other modes

response = openai.chat.completions.create(
    model="gpt-4o",
    messages=[
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_prompt}
    ],
    temperature=0.7 # Slightly creative but consistent
)

return response.choices[0].message.content

```

User Control & Editing

Before Sending:

- Full email preview
- Editable text area (user can modify AI output)
- Regenerate button (if not satisfied)

- Save as draft option

Post-Send:

- Email sent via Gmail API (appears in user's Sent folder)
 - Copy saved in Influo AI for reference
 - Activity logged in deal history
-

Feature 3: Deal Pipeline & Tracking

What It Does

Visual Kanban-style board that tracks every brand deal from initial contact to payment completion.

Pipeline Stages

1. NEW

- Emails just detected by AI
- No action taken yet
- Shows: Brand name, deliverables preview, received date

2. NEGOTIATING

- User has replied with pricing or counter-offer
- Waiting for brand response
- Shows: Quoted amount, last reply date, days since last message

3. ACCEPTED

- Brand agreed to terms
- Contract/brief pending or received
- Shows: Final amount, advance payment status, deadline

4. COMPLETED

- Content posted/delivered
- Awaiting final payment
- Shows: Completion date, payment due date, overdue flag

5. PAID

- Full payment received
- Deal closed successfully
- Shows: Total earned, payment date, deal duration

Additional States:

- **ARCHIVED:** Deals declined or no longer active
 - **ON HOLD:** Paused by brand or creator
-

Deal Card Data Model

json

```
{
  "deal_id": "deal_12345",
  "user_id": "user_67890",
  "brand_name": "Nykaa",
  "sender_email": "partnerships@nykaa.com",
  "sender_name": "Priya Sharma",

  "stage": "accepted",
  "offer_type": "Paid Sponsorship",

  "deliverables": [
    {
      "type": "instagram_reel",
      "quantity": 2,
      "description": "Product unboxing and review"
    },
    {
      "type": "instagram_story",
      "quantity": 3,
      "description": "Behind-the-scenes + swipe-up"
    }
  ],

  "pricing": {
    "quoted_amount": 40000,
    "negotiated_amount": 38000,
    "final_amount": 38000,
    "currency": "INR",
    "breakdown": [
      {"item": "2 Reels", "price": 30000},
      {"item": "3 Stories", "price": 8000}
    ]
  },

  "payment": {
    "terms": "50% advance, 50% on completion",
    "advance_amount": 19000,
    "advance_received": true,
    "advance_received_date": "2025-02-15",
    "balance_amount": 19000,
    "balance_received": false,
    "payment_status": "partial"
  },
}
```

```
"timeline": {
  "shoot_by": "2025-02-28",
  "post_by": "2025-03-05",
  "payment_due": "2025-03-10"
},

"email_thread": {
  "gmail_thread_id": "thread_abc123",
  "message_count": 5,
  "last_message_at": "2025-02-15T10:30:00Z",
  "last_message_from": "brand"
},

"activity_log": [
  {
    "timestamp": "2025-02-10T09:00:00Z",
    "type": "email_received",
    "description": "Initial collaboration email"
  },
  {
    "timestamp": "2025-02-10T14:30:00Z",
    "type": "email_sent",
    "description": "Sent pricing proposal (₹40k)"
  },
  {
    "timestamp": "2025-02-12T11:00:00Z",
    "type": "email_received",
    "description": "Brand countered at ₹38k"
  },
  {
    "timestamp": "2025-02-12T15:00:00Z",
    "type": "email_sent",
    "description": "Accepted ₹38k"
  },
  {
    "timestamp": "2025-02-15T10:30:00Z",
    "type": "payment_received",
    "description": "Advance payment received (₹19k)"
  }
],

"created_at": "2025-02-10T09:00:00Z",
"updated_at": "2025-02-15T10:30:00Z"
}
```

User Actions on Deal Cards

From Any Stage:

- View full email thread
- Add manual note
- Edit deal details
- Move to different stage (manual override)
- Archive deal

Stage-Specific Actions:

NEW:

- Reply with AI
- Mark as spam

NEGOTIATING:

- Send follow-up reminder (if no response in 7 days)
- Move to Accepted (if brand agreed)
- Move to Archived (if declined)

ACCEPTED:

- Mark advance received (updates payment status)
- Add contract/brief (file upload)
- Mark as Completed (when content posted)

COMPLETED:

- Send payment reminder
- Mark balance received
- Move to Paid (auto-moves when balance marked received)

PAID:

- Export invoice/receipt
 - Leave brand review (for future Brand Intel feature)
-

Filtering & Search

Filters:

- Stage (multi-select)
- Offer type (Paid / Barter / Gifted)
- Date range (This month / Last 3 months / All time)
- Payment status (Pending / Partial / Paid)
- Amount range (₹0-10k / ₹10k-50k / ₹50k+)

Search:

- Brand name
- Deal amount
- Deliverable type

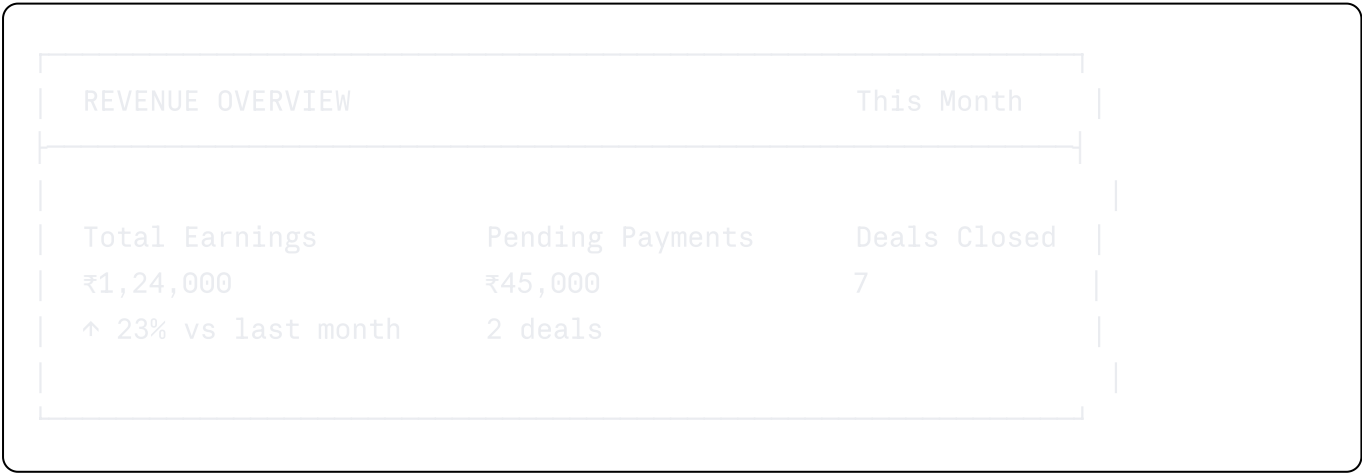
Sorting:

- Newest first
- Highest amount
- Closest deadline
- Overdue payments

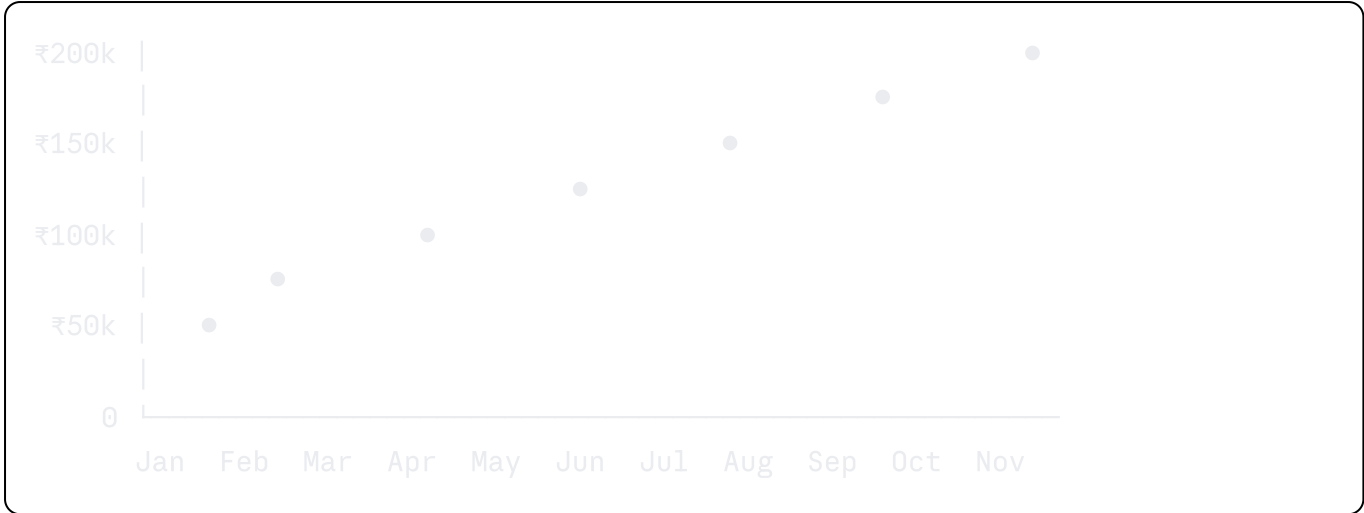
Feature 4: Dashboard & Analytics

What It Shows

Revenue Overview:



Monthly Breakdown (Chart):



Deal Distribution:

Paid Sponsorships:	65% (13 deals)	₹2,60,000
Barter Collaborations:	25% (5 deals)	₹0
Gifted Products:	10% (2 deals)	₹0

Top Brands:

1. Nykaa	- 3 deals	- ₹1,15,000
2. Myntra	- 2 deals	- ₹95,000
3. Boat	- 2 deals	- ₹50,000

Average Deal Value:

All Deals:	₹19,500
Paid Only:	₹28,000
This Month:	₹17,700 (trending down, negotiate harder!)

Feature 5: Benchmark Pricing Suggestions

What It Does

Provides pricing guidance based on creator's follower count and engagement rate using industry benchmarks.

Pricing Formula

Base Rate Calculation:

```
python
```

```

def suggest_pricing(followers, engagement_rate, content_type, platform="instagram")
    """
    Calculates suggested pricing for content
    """

    # Industry benchmark rates (India, 2025)
    # Per 10k followers base rate

    base_rates = {
        "instagram": {
            "reel": 3500,
            "story": 800,
            "post": 2500,
            "carousel": 3000
        },
        "youtube": {
            "dedicated_video": 15000,
            "integration": 8000,
            "shorts": 4000
        }
    }

    # Engagement multiplier
    if engagement_rate >= 7:
        engagement_multiplier = 1.5 # High engagement
    elif engagement_rate >= 5:
        engagement_multiplier = 1.3
    elif engagement_rate >= 3:
        engagement_multiplier = 1.15
    elif engagement_rate >= 1.5:
        engagement_multiplier = 1.0
    else:
        engagement_multiplier = 0.85 # Low engagement penalty

    # Follower tier multiplier (economies of scale)
    follower_tiers = [
        (10000, 1.0),
        (50000, 0.95),
        (100000, 0.9),
        (500000, 0.85),
        (1000000, 0.8)
    ]

    tier_multiplier = 1.0

```

```

for threshold, multiplier in follower_tiers:
    if followers >= threshold:
        tier_multiplier = multiplier

# Calculate
follower_units = followers / 10000
base_price = base_rates[platform][content_type]

suggested_price = (
    base_price *
    follower_units *
    engagement_multiplier *
    tier_multiplier
)

# Round to nearest ₹500
return round(suggested_price / 500) * 500

# Examples:
# 50k followers, 6% engagement, Instagram Reel:
# ₹3500 * 5 * 1.3 * 0.95 = ₹21,612 → ₹22,000

# 150k followers, 4% engagement, YouTube integration:
# ₹8000 * 15 * 1.15 * 0.9 = ₹1,24,200 → ₹1,24,000

```

UI Presentation

When Generating Reply:

Suggested Pricing	
Based on 52k followers, 6.2% engagement	
Instagram Reel	₹18,000 - ₹22,000
Instagram Story (per story)	₹3,000 - ₹4,000
Carousel Post	₹12,000 - ₹15,000
💡 Tip: Your engagement (6.2%) is above average. You can negotiate toward the higher end.	
[Use These Rates]	[Customize]

Customization Modal:

Customize Pricing

Content Type	Quantity	Price Each	Total
Instagram Reel	2	₹ [20,000]	₹40,000
Instagram Story	3	₹ [3,500]	₹10,500
Carousel Post	1	₹ [12,000]	₹12,000
Package Total:			₹62,500

[+] Add deliverable

Additional Terms:

☒ Usage rights for 6 months included

☒ 50% advance payment required

☐ Exclusivity (no competing brands for 30 days)

[Save & Use in Reply]

Limitations & Transparency

What We Tell Users:

 Pricing Disclaimer

These suggestions are based on industry benchmarks and your follower count.

Actual rates vary based on:

- Brand budget and campaign scope
- Your content quality and niche authority
- Seasonal demand (festival campaigns pay more)
- Exclusivity and usage rights

Use these as a starting point, not final rates.

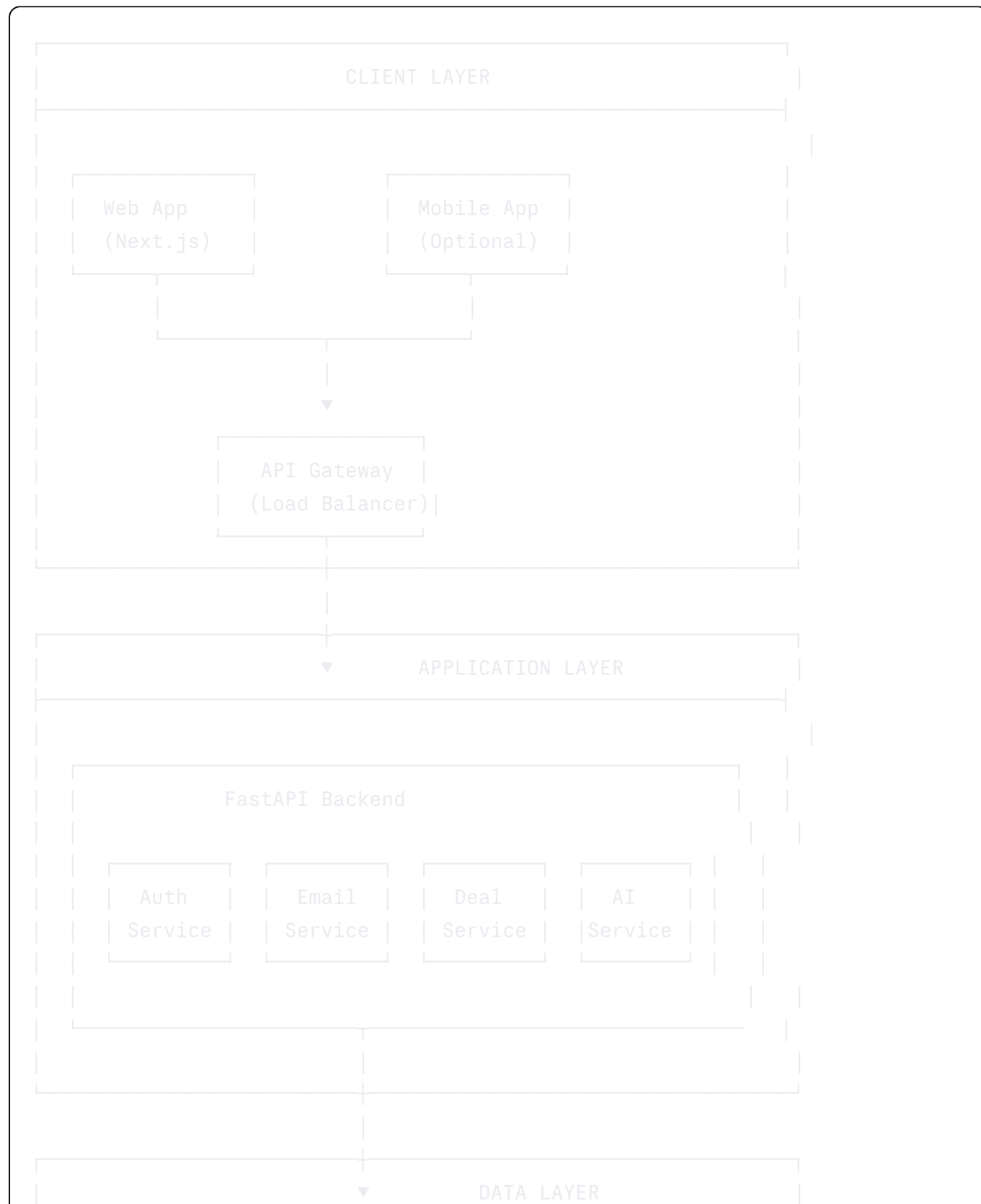
Data Sources (Transparently Communicated):

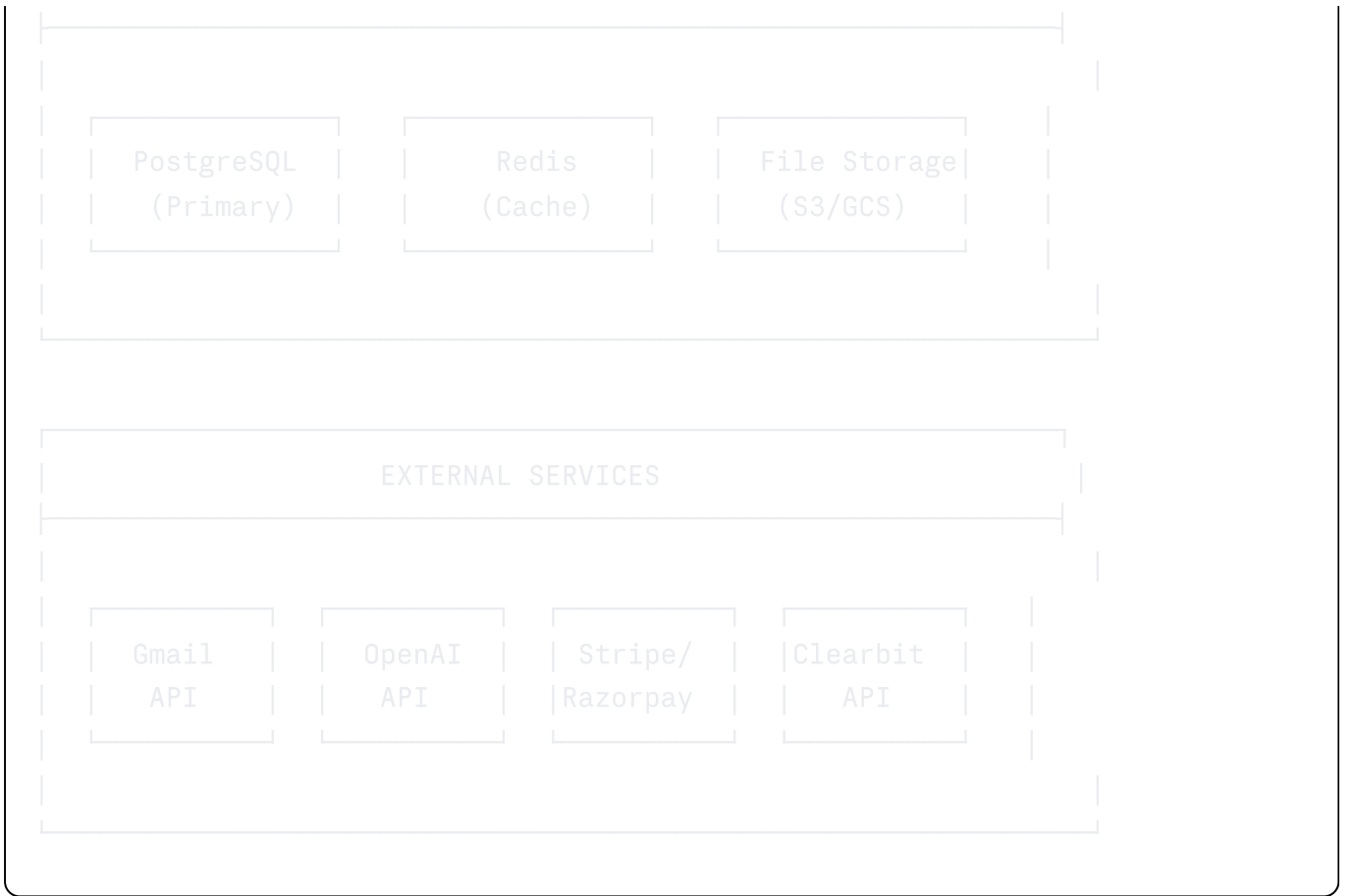
- Industry reports (INCA Influencer Marketing Report 2024)
- Public creator rate cards (where available)

- Aggregated data from creator communities
 - NOT personalized to individual brands (we don't have that data yet)
-

6. Technical Architecture

System Architecture Diagram





Tech Stack Breakdown

Frontend

- **Framework:** Next.js 14 (App Router)
- **UI Library:** React 18
- **Styling:** Tailwind CSS + shadcn/ui components
- **State Management:** Zustand (lightweight, simpler than Redux)
- **API Client:** Axios + React Query (caching, optimistic updates)
- **Auth:** NextAuth.js (Google OAuth)

Backend

- **Framework:** FastAPI (Python 3.11+)
- **API Design:** RESTful + WebSocket (for real-time updates)
- **Background Jobs:** Celery + Redis (email sync, AI processing)
- **Email Queue:** Gmail API + Pub/Sub webhooks

Database

- **Primary DB:** PostgreSQL 15

- ACID compliance for financial data (payments)
- JSON fields for flexible deal metadata
- Full-text search for email content
- **Cache Layer:** Redis
 - Session storage
 - API response caching
 - Rate limiting

AI Layer

- **LLM Provider:** OpenAI API
 - Model: GPT-4o (email classification, reply generation)
 - Model: GPT-4o-mini (quick keyword extraction, cheap calls)
- **Embeddings:** (Future) For semantic email search

Infrastructure

- **Hosting:** Railway / Render (MVP), AWS (scale)
 - **CDN:** Cloudflare (static assets, DDoS protection)
 - **File Storage:** AWS S3 / Google Cloud Storage (contracts, invoices)
 - **Monitoring:** Sentry (error tracking), Posthog (analytics)
-

API Architecture

Core Endpoints

Authentication:

POST	/auth/google/login	# Initiate Google OAuth
GET	/auth/google/callback	# OAuth callback
POST	/auth/logout	# Logout
GET	/auth/me	# Get current user

Email Management:

Deal Management:

GET	/deals	# List all deals (with filters)
GET	/deals/{deal_id}	# Get deal details
PATCH	/deals/{deal_id}	# Update deal (stage, payment, notes)
POST	/deals/{deal_id}/archive	# Archive deal
GET	/deals/stats	# Revenue stats for dashboard

AI Services:

POST	/ai/classify-email	# Classify if email is brand deal
POST	/ai/extract-deal-data	# Extract structured data from email
POST	/ai/generate-reply	# Generate email reply
POST	/ai/suggest-pricing	# Get pricing suggestions

Webhooks:

POST	/webhooks/gmail	# Gmail push notification endpoint
POST	/webhooks/stripe	# Stripe payment webhook (future)

Database Schema

Users Table

sql

```

CREATE TABLE users (
  user_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email VARCHAR(255) UNIQUE NOT NULL,
  name VARCHAR(255),

  -- OAuth tokens (encrypted)
  google_access_token TEXT,
  google_refresh_token TEXT,
  token_expires_at TIMESTAMP,

  -- Profile
  platform VARCHAR(50), -- 'instagram', 'youtube', 'both'
  instagram_handle VARCHAR(100),
  youtube_channel VARCHAR(100),
  followers_instagram INTEGER,
  followers_youtube INTEGER,
  engagement_rate_instagram DECIMAL(5,2),
  engagement_rate_youtube DECIMAL(5,2),
  content_niche VARCHAR(50),

  -- Settings
  email_sync_enabled BOOLEAN DEFAULT TRUE,
  notification_preferences JSONB,

  -- Subscription
  subscription_tier VARCHAR(50) DEFAULT 'free',
  subscription_expires_at TIMESTAMP,

  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_users_email ON users(email);

```

Deals Table

```
sql
```

```

CREATE TABLE deals (
  deal_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(user_id) ON DELETE CASCADE,

  -- Brand info
  brand_name VARCHAR(255) NOT NULL,
  brand_domain VARCHAR(255),
  sender_email VARCHAR(255) NOT NULL,
  sender_name VARCHAR(255),

  -- Deal details
  stage VARCHAR(50) NOT NULL, -- 'new', 'negotiating', 'accepted', 'complete
  offer_type VARCHAR(50),      -- 'Paid Sponsorship', 'Barter', 'Gifted', 'Af

  deliverables JSONB, -- [{"type": "reel", "quantity": 2, "description": "..

  -- Pricing
  quoted_amount INTEGER,      -- Amount creator first quoted
  negotiated_amount INTEGER,  -- Final agreed amount
  final_amount INTEGER,       -- Actual amount (may differ due to adjustments
  currency VARCHAR(10) DEFAULT 'INR',
  pricing_breakdown JSONB,

  -- Payment
  payment_terms VARCHAR(255),
  advance_amount INTEGER,
  advance_received BOOLEAN DEFAULT FALSE,
  advance_received_date DATE,
  balance_amount INTEGER,
  balance_received BOOLEAN DEFAULT FALSE,
  balance_received_date DATE,
  payment_status VARCHAR(50), -- 'pending', 'partial', 'paid'

  -- Timeline
  shoot_by DATE,
  post_by DATE,
  payment_due_date DATE,

  -- Email thread
  gmail_thread_id VARCHAR(255),
  gmail_message_ids TEXT[],
  last_email_at TIMESTAMP,
  last_email_from VARCHAR(50), -- 'creator' or 'brand'

```

```

-- Metadata
notes TEXT,
tags TEXT[],

-- Timestamps
created_at TIMESTAMP DEFAULT NOW(),
updated_at TIMESTAMP DEFAULT NOW(),
accepted_at TIMESTAMP,
completed_at TIMESTAMP,
paid_at TIMESTAMP
);

CREATE INDEX idx_deals_user_stage ON deals(user_id, stage);
CREATE INDEX idx_deals_gmail_thread ON deals(gmail_thread_id);
CREATE INDEX idx_deals_created_at ON deals(created_at DESC);

```

Activity Log Table

```

sql

CREATE TABLE deal_activities (
  activity_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  deal_id UUID REFERENCES deals(deal_id) ON DELETE CASCADE,
  user_id UUID REFERENCES users(user_id) ON DELETE CASCADE,

  activity_type VARCHAR(50) NOT NULL,
  -- Types: 'email_received', 'email_sent', 'stage_changed',
  --         'payment_received', 'note_added', 'deal_created'

  description TEXT,
  metadata JSONB, -- Flexible data for each activity type

  created_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_activities_deal ON deal_activities(deal_id, created_at DESC);

```

Email Cache Table

```

sql

```

```

CREATE TABLE email_cache (
  email_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(user_id) ON DELETE CASCADE,

  gmail_message_id VARCHAR(255) UNIQUE NOT NULL,
  gmail_thread_id VARCHAR(255),

  sender_email VARCHAR(255),
  sender_name VARCHAR(255),
  subject TEXT,
  body_text TEXT,
  body_html TEXT,
  received_at TIMESTAMP,

  -- Classification
  is_brand_email BOOLEAN,
  classification_confidence INTEGER, -- 0-100

  -- Extracted data
  extracted_data JSONB,

  -- Deal association
  deal_id UUID REFERENCES deals(deal_id),

  created_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_email_cache_user ON email_cache(user_id, received_at DESC);
CREATE INDEX idx_email_cache_gmail_id ON email_cache(gmail_message_id);
CREATE INDEX idx_email_cache_thread ON email_cache(gmail_thread_id);

```

7. Data Models

Deal State Machine



State Transitions:

From	To	Trigger	Validation
NEW	NEGOTIATING	User sends reply	-
NEW	ARCHIVED	User marks as spam/declines	-
NEGOTIATING	ACCEPTED	Brand confirms deal	<code>final_amount</code> must be set
NEGOTIATING	ARCHIVED	Deal falls through	-
ACCEPTED	COMPLETED	Content delivered	<code>completed_at</code> timestamp set
ACCEPTED	ARCHIVED	Deal canceled	-
COMPLETED	PAID	Payment received	<code>balance_received = true</code>
COMPLETED	COMPLETED	Still waiting payment	Payment reminder sent

Payment Status Logic

```
python
```

```

def calculate_payment_status(deal):
    """
    Determines payment status based on received amounts
    """

    if not deal.advance_received and not deal.balance_received:
        return "pending"

    elif deal.advance_received and not deal.balance_received:
        return "partial"

    elif deal.advance_received and deal.balance_received:
        return "paid"

    else:
        # Edge case: balance received but not advance
        # (unlikely but possible if terms were different)
        return "paid"

def check_payment_overdue(deal):
    """
    Checks if payment is overdue
    """

    if deal.stage != "completed":
        return False

    if deal.payment_status == "paid":
        return False

    # If no payment_due_date set, assume 7 days after completion
    due_date = deal.payment_due_date or (deal.completed_at + timedelta(days=7))

    return datetime.now() > due_date

```

8. AI System Design

Email Classification Pipeline Step 1: Quick Keyword Filter

python

```

BRAND_EMAIL_KEYWORDS = [
    # Direct collaboration terms
    "collaboration", "partnership", "sponsorship", "brand deal",
    "paid promotion", "sponsored post", "influencer campaign",

    # Role indicators
    "brand manager", "marketing team", "PR agency", "creator partnerships",

    # Deliverable mentions
    "instagram reel", "youtube video", "sponsored content", "product review",

    # Commercial terms
    "budget", "compensation", "rates", "payment terms", "invoice"
]

def quick_keyword_check(email_subject, email_body):
    """
    Fast filter to skip obviously non-brand emails
    Saves AI API calls
    """
    text = f"{email_subject} {email_body}".lower()

    # Negative keywords (auto-reject)
    if any(word in text for word in ["unsubscribe", "verify your email", "reset
        return False

    # Positive keywords
    keyword_matches = sum(1 for keyword in BRAND_EMAIL_KEYWORDS if keyword in t

    # If 2+ keywords found, likely a brand email
    return keyword_matches >= 2

```

Step 2: LLM Classification

python

```

def classify_with_ai(email_data):
    """
    Uses GPT-4o-mini for accurate classification
    """

    prompt = f"""
    Analyze this email and determine if it's a brand collaboration request.

    FROM: {email_data['sender_email']}
    SUBJECT: {email_data['subject']}
    BODY PREVIEW: {email_data['body'][:1000]}

    Brand collaboration emails are:
    - Sent by brands, marketing agencies, or PR firms
    - Proposing paid sponsorships, barter deals, or gifted products
    - Asking about creator rates, availability, or interest

    NOT brand emails:
    - Job offers to work as an employee
    - Sales emails selling software/services to the creator
    - Fan emails from followers
    - Newsletter subscriptions
    - Spam/phishing

    Return JSON:
    {{
        "is_brand_email": true or false,
        "confidence": 0-100,
        "reasoning": "brief explanation",
        "suggested_action": "reply" or "archive" or "spam"
    }}
    """

    response = openai.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": prompt}],
        response_format={"type": "json_object"},
        temperature=0.3 # Low temp for consistent classification
    )

    return json.loads(response.choices[0].message.content)

```


Data Extraction System

Structured Extraction with Function Calling:

python

```

def extract_deal_data(email_body):
    """
    Extracts structured deal information from email
    """

    extraction_schema = {
        "type": "object",
        "properties": {
            "brand_name": {
                "type": "string",
                "description": "Company or brand name sending the email"
            },
            "offer_type": {
                "type": "string",
                "enum": ["Paid Sponsorship", "Barter", "Gifted Product", "Affil
            "description": "Type of collaboration offer"
            },
            "deliverables": {
                "type": "array",
                "items": {
                    "type": "object",
                    "properties": {
                        "content_type": {"type": "string"},
                        "quantity": {"type": "integer"},
                        "description": {"type": "string"}
                    }
                },
                "description": "List of content deliverables requested"
            },
            "budget_mentioned": {
                "type": "boolean",
                "description": "Whether a budget amount is mentioned in the ema
            },
            "budget_amount": {
                "type": ["number", "null"],
                "description": "Proposed budget in INR, or null if not mentione
            },
            "deadline": {
                "type": ["string", "null"],
                "format": "date",
                "description": "Content deadline in YYYY-MM-DD format"
            },
            "sender_role": {
                "type": "string",

```

```

        "description": "Role/title of sender (e.g., 'Marketing Manager'
    },
    "additional_requirements": {
        "type": "array",
        "items": {"type": "string"},
        "description": "Other requirements (exclusivity, usage rights,
    }
},
"required": ["brand_name", "offer_type", "deliverables"]
}

messages = [
    {
        "role": "system",
        "content": "You are a data extraction assistant. Extract brand coll
    },
    {
        "role": "user",
        "content": f"Extract deal information from this email:\n\n{email_bo
    }
]

response = openai.chat.completions.create(
    model="gpt-4o",
    messages=messages,
    functions=[{
        "name": "extract_deal_data",
        "description": "Extract structured brand deal information",
        "parameters": extraction_schema
    }],
    function_call={"name": "extract_deal_data"}
)

function_call = response.choices[0].message.function_call
return json.loads(function_call.arguments)

```

Reply Generation System

Context-Aware Prompting:

python

```

def build_reply_context(user_profile, deal_data, reply_mode):
    """
    Builds comprehensive context for AI reply generation
    """

    context = {
        "creator_info": {
            "name": user_profile.name,
            "platform": user_profile.platform,
            "followers": user_profile.followers,
            "engagement_rate": user_profile.engagement_rate,
            "niche": user_profile.content_niche,
            "past_brands": get_past_brand_names(user_profile.user_id)
        },
        "deal_context": {
            "brand_name": deal_data.brand_name,
            "deliverables": deal_data.deliverables,
            "budget_mentioned": deal_data.budget_mentioned,
            "deadline": deal_data.deadline
        },
        "conversation_history": get_email_thread(deal_data.gmail_thread_id),
        "reply_intent": reply_mode
    }

    return context


def generate_reply_prompt(context, custom_instructions):
    """
    Creates tailored prompt for each reply mode
    """

    base_instructions = f"""
    You are writing an email on behalf of {context['creator_info']['name']},
    a {context['creator_info']['niche']} content creator with
    {context['creator_info']['followers']} followers.

    TONE REQUIREMENTS:
    - Professional but warm (not corporate)
    - Confident without arrogance
    - Enthusiastic but not desperate
    - Clear and direct (no fluff)

    STRUCTURE:
    - Personalized greeting (use sender's name if available)
  
```

- Acknowledge their message specifically
- Deliver main message based on intent
- Clear next steps
- Professional sign-off

AVOID:

- Generic openings ("I hope this email finds you well")
- Over-enthusiasm ("I'm SO EXCITED!!!")
- Underselling ("I'm just a small creator")
- Vague language ("maybe we can discuss")

"""

```
if context['reply_intent'] == "accept_and_price":
```

```
    specific_instructions = f"""
```

```
    TASK: Accept the collaboration and present your pricing.
```

```
    DELIVERABLES TO QUOTE:
```

```
    {format_deliverables(context['deal_context']['deliverables'])}
```

```
    PRICING TO PRESENT:
```

```
    {format_pricing_breakdown(context['pricing_data'])}
```

```
    INCLUDE:
```

- Express genuine interest in the campaign/brand
- List deliverables clearly (itemized)
- Present pricing breakdown (transparent)
- State payment terms: 50% advance, 50% on completion
- Mention usage rights: 6 months included
- Suggest next step: call/meeting to discuss details

```
    {custom_instructions}
```

"""

```
elif context['reply_intent'] == "negotiate":
```

```
    specific_instructions = f"""
```

```
    TASK: Negotiate the offer professionally.
```

```
    THEIR OFFER: ₹{context['brand_offer']}
```

```
    YOUR TARGET: ₹{context['target_price']}
```

```
    APPROACH:
```

- Acknowledge their offer warmly
- Explain your value without being defensive:
 - * Your engagement rate ({context['creator_info']['engagement_rate']})%
 - * Quality of your content

- * Audience alignment with their brand
- Present counter-offer with justification
- Offer alternative: "If budget is ₹X, I can do [reduced scope]"
- Keep tone collaborative, not adversarial

```
{custom_instructions}  
"""  
  
# ... other reply modes  
  
return base_instructions + "\n\n" + specific_instructions
```

Quality Control:

python

```
def validate_generated_reply(reply_text, deal_data):
    """
    Checks AI-generated reply for quality issues
    """

    issues = []

    # Check length (not too short, not too long)
    if len(reply_text) < 100:
        issues.append("Reply too short (under 100 chars)")
    if len(reply_text) > 2000:
        issues.append("Reply too long (over 2000 chars)")

    # Check for placeholder text
    placeholders = ["[BRAND NAME]", "[YOUR NAME]", "[INSERT", "TODO"]
    if any(ph in reply_text for ph in placeholders):
        issues.append("Contains placeholder text")

    # Check pricing format (if pricing mode)
    if "₹" in reply_text:
        # Ensure pricing is present and formatted
        if not re.search(r"₹\s*[\d,]+", reply_text):
            issues.append("Pricing format unclear")

    # Check for overly generic phrases
    generic_phrases = [
        "I hope this email finds you well",
        "Thank you for your email",
        "I look forward to hearing from you"
    ]
    generic_count = sum(1 for phrase in generic_phrases if phrase.lower() in re
    if generic_count >= 2:
        issues.append("Too many generic phrases")

    # Check for excessive exclamation marks
    if reply_text.count("!") > 3:
        issues.append("Excessive exclamation marks")

    return issues
```

9. Security & Privacy

Data Protection

Sensitive Data Handling:

1. OAuth Tokens:

- Encrypted at rest using AES-256
- Stored in PostgreSQL with encryption
- Never logged or exposed in API responses
- Rotated every 7 days via refresh token

2. Email Content:

- Email bodies stored temporarily (30 days max)
- Full-text search uses PostgreSQL tsvector (not plaintext index)
- User can delete email cache anytime

3. Payment Data:

- No credit card storage (handled by Stripe/Razorpay)
- Deal amounts stored as integers (prevent float errors)
- Payment dates and amounts logged for transparency

GDPR Compliance:

- **Right to Access:** User can export all their data (JSON download)
 - **Right to Delete:** Account deletion removes all associated data
 - **Right to Portability:** Export deals as CSV/JSON
 - **Consent:** Clear OAuth permission explanations
-

Gmail API Security

Scopes Requested:

<code>https://www.googleapis.com/auth/gmail.readonly</code>	# Read emails
<code>https://www.googleapis.com/auth/gmail.send</code>	# Send replies
<code>https://www.googleapis.com/auth/gmail.modify</code>	# Add labels

Security Practices:

- Never request `gmail.full` scope (too broad)

- Only fetch emails from last 3 months on initial sync
- Rate limiting: Max 100 emails fetched per minute
- Webhook verification: HMAC signature validation

Error Handling:

- Token expiration: Auto-refresh or prompt re-auth
 - Quota exceeded: Retry with exponential backoff
 - Permission revoked: Graceful degradation (notify user)
-

AI Safety

Prompt Injection Prevention:

```
python

def sanitize_user_input(custom_instructions):
    """
    Prevents user from injecting malicious prompts
    """

    # Blacklist dangerous patterns
    dangerous_patterns = [
        "ignore previous instructions",
        "disregard system prompt",
        "you are now",
        "new role:",
        "forget everything"
    ]

    sanitized = custom_instructions
    for pattern in dangerous_patterns:
        if pattern.lower() in sanitized.lower():
            # Remove or escape
            sanitized = sanitized.replace(pattern, "[REMOVED]")

    # Limit length
    if len(sanitized) > 500:
        sanitized = sanitized[:500] + "..."

    return sanitized
```

Content Moderation:

- AI-generated replies scanned for:
 - Inappropriate language
 - Discriminatory content
 - Spam-like patterns
 - Flagged replies require manual review before sending
-

10. Success Metrics

Product Metrics

Activation (Week 1):

- Gmail connected: 100%
- At least 1 brand email detected: 70%
- First AI reply generated: 50%

Engagement (Monthly):

- Active users (logged in 2+ times): 60%
- Emails classified: 50+ per user
- AI replies sent: 3+ per user
- Deals moved to "Accepted" stage: 1+ per user

Retention:

- Day 7 retention: 40%
- Day 30 retention: 25%
- Month 3 retention: 15%

Revenue Impact (Creator-reported):

- Deals closed via Influo AI: 5+ per month (power users)
 - Earnings tracked: ₹20k-₹50k per month
 - Payment recovery: 1-2 overdue payments nudged per month
-

Technical Metrics

Performance:

- API response time: <200ms (p95)
- Gmail sync: <3 minutes for 1000 emails
- AI reply generation: <5 seconds
- Uptime: 99.5%+

AI Quality:

- Email classification accuracy: 90%+
- False positive rate: <5%
- Reply acceptance rate (user sends without editing): 60%+

Cost Efficiency:

- OpenAI API cost per user: <₹50/month
 - Gmail API quota usage: <80% of daily limit
-

Business Metrics

Conversion (Free → Paid):

- Trial to paid: 10–15%
- Churn rate: <5% monthly

CAC & LTV:

- Customer Acquisition Cost: ₹500–₹1000 (organic + ads)
- Lifetime Value: ₹6000+ (12 month retention at ₹500/month)
- LTV:CAC ratio: 6:1+

NPS (Net Promoter Score):

- Target: 40+ (good for B2C SaaS)
-

Summary

What Inflow AI Does

Solves 4 core creator problems:

1. **Brand emails lost in chaos** → AI auto-detection

2. **Manual reply writing** → One-click AI responses
3. **No deal visibility** → Visual pipeline tracking
4. **Pricing uncertainty** → Benchmark suggestions

MVP Features (Ship First)

- ✓ Smart Brand Deal Inbox
- ✓ AI Reply Generation (4 modes)
- ✓ Deal Pipeline (5 stages)
- ✓ Basic Pricing Suggestions
- ✓ Payment Tracking

NOT in MVP

- ✗ Brand Intelligence (need data first)
- ✗ Invoice Generation (not core problem)
- ✗ Multi-platform inbox (focus Gmail only)
- ✗ Advanced analytics (ship later)

Success Looks Like

Month 1:

- 100 users signed up
- 50 users send their first AI reply
- 20 users close a deal tracked in Inflow AI

Month 3:

- 500 users
- 100 paying users (₹499-₹999/month)
- ₹50k MRR
- NPS 40+

Month 6:

- 2000 users
 - 400 paying users
 - ₹2L MRR
 - Clear product-market fit signal (low churn, high engagement)
-

Next Steps

1. **Design mockups** (landing page + app UI)
 2. **Build backend** (FastAPI + PostgreSQL + Gmail integration)
 3. **Build frontend** (Next.js + core workflows)
 4. **Alpha test** (10 creators, manual onboarding)
 5. **Beta launch** (waitlist → 100 users)
 6. **Iterate based on feedback** (fix what's broken, double down on what works)
-

End of Document